A.春

A-春_牛客练习赛132

原题

题目

在坐标轴 1 到 n 的 n 个位置上

有长度为 a_i 的 n 根木棒

其下会围成一个不规则多边形（详见样例）

你可以多次交换不同相邻的木棒，求其下最大覆盖面积 S

输入描述

第一行输入 n

接下来输入一行包括 n 个正整数 a_i ($1 \le n \le 2 \times 10^6; 1 \le a_i \le 2 \times 10^6$)

输出描述

请输出 n 根木棒的最大围成面积 S

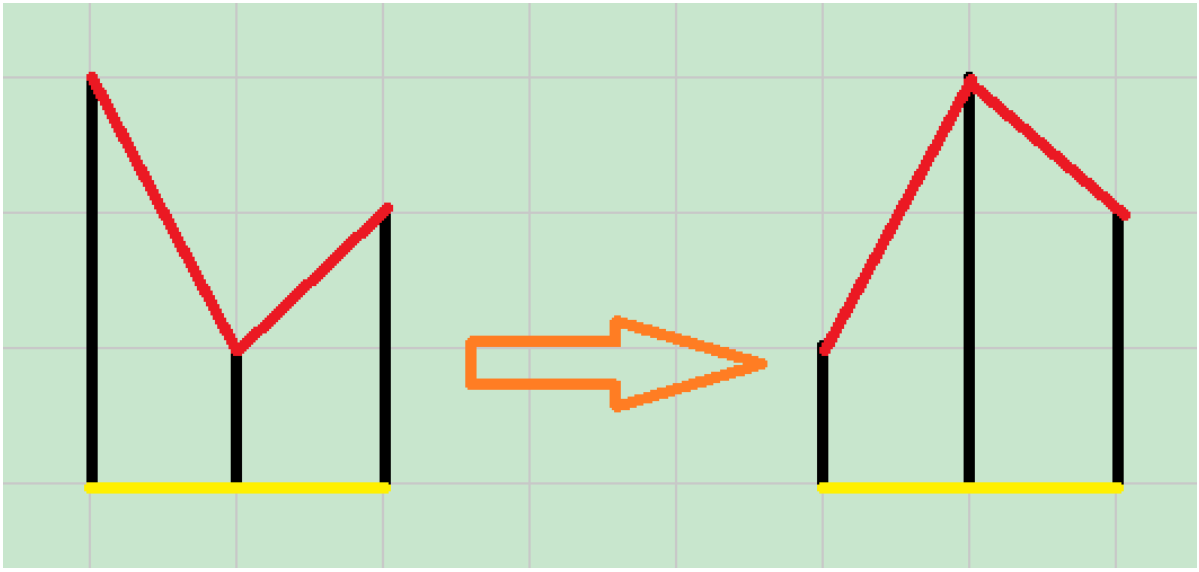
为了输出方便，请**四舍五入**后保留两位小数

示例

输入
3
3 1 2

输出
4.50

说明



编辑

交换第1、2根木棒，变成 [1, 3, 2]

面积从： (3+1)*1/2+(1+2)*1/2=3.5

变为：(1+3)*1/2+(3+2)/2=4.5

不存在更大的面积

题解

思路1

为了使面积最大，我们应当让大的木棍尽可能的靠在一起。

对木棍进行排序，取出最大的两个木棍，设为l, r， 并重复以下操作

若果 l > r，此时将新的木棍放在左边的收益最大， 更新l为新的木棍

否则， 将新的木棍放在右边的收益最大， 更新r为新的木棍

每段的面积和即为所求值

时间复杂度o(nlogn)

代码如下

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

inline ll read(){
    ll s = 0, w = 1; char ch = getchar();
    while (ch < 48 || ch > 57) { if (ch == '-') w = -1; ch = getchar(); }
    while (ch >= 48 && ch <= 57) s = (s << 1) + (s << 3) + (ch ^ 48), ch =
getchar();
    return s * w;
}

bool cmp(ll x, ll y){
    return x>y;
}

void solve(){
    ll n = read();
    vector<ll> a(n);
    for(ll i=0;i<n;i++){
        a[i] = read();
    }
    if (n==1){
        printf("%.2f\n", 0);
        return ;
    }
    sort(a.begin(), a.end(), cmp);
    ll l=a[0], r=a[1];
    double ans=(l+r)*1.0/2;
    for(ll t=2;t<n;t++){
        if (l>r){
            ans += (l+a[t]) * 1.0/2;
```

```
            l = a[t];
        }
        else{
            ans += (r+a[t]) * 1.0/2;
            r = a[t];
        }
    }
    printf("%.2f\n", ans);
}


int main(){
    //ll t = read();
    ll t = 1;
    while (t--){
        solve();
    }
}
```

思路2（比赛的时候没想到）

考虑某种排序下木棍组成的图形的面积（$s_1, s_2, \ldots, s_n$）

$$S = \frac{(s_1 s_2)\times 1}{2}\frac{(s_2 s_3)\times 1}{2}\ldots\frac{(s_{n-1} s_n)\times 1}{2} = \frac{s_1 s_n}{2} s_2 s_3 \ldots . s_{n-1}$$

发现只有边缘的木棍只出现一次，贡献最小

只需将最小的两根木棍放在两边面积即为最大值

时间复杂度o(nlogn)

代码如下

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

inline ll read(){
    ll s = 0, w = 1; char ch = getchar();
    while (ch < 48 || ch > 57) { if (ch == '-') w = -1; ch = getchar(); }
    while (ch >= 48 && ch <= 57) s = (s << 1) + (s << 3) + (ch ^ 48), ch =
getchar();
    return s * w;
}

void solve(){
    ll n = read();
    vector<ll> a(n);
    for(ll i=0;i<n;i++){
        a[i] = read();
    }
    if (n==1){
        printf("%.2f\n", 0);
        return ;
    }
    sort(a.begin(), a.end());
```

```
    double ans = (a[0] + a[1])/2.0;
    for(ll i=2;i<n;i++){
       ans += a[i];
    }
    printf("%.2f\n", ans);
}


int main(){
    //ll t = read();
    ll t = 1;
    while (t--){
        solve();
    }
}
```

B.江

[B-江_牛客练习赛132]

原题

题目

在斗地主中连续的一组数组叫顺子，如$[3,4,5,6,7],[9,8,7,6,5]$;

在这里为了方便将$1,2$也记入顺子中，且顺子长度没有任何限制，如$[1,2,3],[5]$均是顺子

可这样求最长顺子长度太无趣了...

于是聪明的你引入了鬼牌的概念、鬼牌可以变为 1 到 m 中任何你想要的牌

好了，现在求可以组成的最长顺子长度

-赛后补充:

题目中最长顺子中的牌可以是原本中任意一张牌or鬼牌，组成的顺子必须连续无中断（排序后为公差为1的等差数列）

输入描述

第一行输入 n,m,k，分别表示有 n 张牌介于 1 到 m 之间，另外有 k 张鬼牌 (共n+k张牌)

第二行输入 n 个正整数a_i，表示 n 张牌的点数

$$1 \le n \le 5 \times 10^5; 1 \le m \le 2 \times 10^9; 0 \le k \le 2 \times 10^9; 1 \le a_i \le m$$

输出描述

输出一个非负整数表示可行顺子的最大长度

题解

思路一

考虑到原来的牌中重复的对最长顺子无意义，因此先去重

手牌可以视为由一段段联系的牌组成， 即一系列 b_{l_i}, b_{r_i}

使用滑动窗口遍历b数组， 不断使用鬼牌填补两段区间的空隙， 如果超出了k，则收缩左指针。

对一段使用了g张鬼牌填补的区间l, r， 由其扩展的最大可能顺子长度为$min(r - l1k - g, n)$（不考虑合并其他区间的情况下）

代码如下

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
typedef pair<long long,long long> PLL;
inline ll read(){
    ll s = 0, w = 1; char ch = getchar();
    while (ch < 48 || ch > 57) { if (ch == '-') w = -1; ch = getchar(); }
    while (ch >= 48 && ch <= 57) s = (s << 1) + (s << 3) + (ch ^ 48), ch =
getchar();
    return s * w;
}

inline void pt(ll x){if(x<0) putchar('-'),x=-x;if(x>9)
pt(x/10);putchar(x%10+'0');}

void print(ll x){pt(x), puts("");}

int main() {
    ll n, m, k;
    n = read();
    m = read();
    k = read();
    vector<ll> a(n);
    for (ll i = 0; i < n; i++) {
        a[i] = read();
    }
    sort(a.begin(), a.end());
    a.erase(unique(a.begin(), a.end()), a.end());
    vector<PLL> b;
    ll l = 0;
    for(ll r=1;r<a.size();r++){
        if ((a[r]-a[r-1])!=1){
            b.push_back({a[l], a[r-1]});
            l = r;
        }
    }
    b.push_back({a[l], a[n-1]});
    if (b.size()==1){
        print(min(b[0].second - b[0].first + 1 + k, m));
    }
    else{
```

```
        ll current_width = b[0].second - b[0].first + 1;
        ll ans = min(current_width + k, m);
        ll l=0;
        ll g_num = 0;
        for(ll r=1;r<b.size();r++){
            g_num += b[r].first - b[r-1].second - 1;
            while(g_num > k&&l<=r){
                g_num -= b[l+1].first - b[l].second - 1;
                l++;
            }
            ans = max(ans, min(b[r].second - b[l].first + 1 + k - g_num, m));
        }
        print(ans);
    }


    return 0;
}
```

C.水

[C-水_牛客练习赛132]

原题

题目

圆上有n等分点

你即将给其中个k点染上红色。

如果染色后，存在两个红点，其连线能平分圆的面积，则认为这种染色方式是美丽的；

请问，对k个点染色有多少种不同染色方式能画出一个美丽的图？

其中对n个点标记为$1, 2, 3...n$；如果两个圆A B染色不同，当且仅当存在1个点在A中被染为红色，而图B中没有染色

请将最终结果对$10^97(1000000007)$取模

备注

圆上n等分点的定义是：在圆的周长上均匀地选择n个点，使得相邻两个点之间的弧长相等。

题解

两点连线能平分圆的面积， 则两点的连线过圆心，即关于圆心对称。

所以当n为奇数时，一定找不到满足条件的两个点。

考虑n为偶数的情况

当k<2时，结果为0

当$k > \frac{n}{2}$时, 一定能找到两个点满足要求， 结果即为从n个位置中选k个的数量， 即C_n^k

当$k \leqslant \frac{n}{2}$时，考虑不满足的个数，即两个点不在同一条过圆心的线上。先选k条线，有$C_{\frac{n}{2}}^k$种，

再排列k个点，有2^k种。所以满足的个数为$C_n^k - C_{\frac{n}{2}}^k \times 2^k$。

为了压缩求C_n^k的时间，预处理n!和其逆元

代码如下

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
typedef pair<long long,long long> PLL;
inline ll read(){
    ll s = 0, w = 1; char ch = getchar();
    while (ch < 48 || ch > 57) { if (ch == '-') w = -1; ch = getchar(); }
    while (ch >= 48 && ch <= 57) s = (s << 1) + (s << 3) + (ch ^ 48), ch =
getchar();
    return s * w;
}

inline void pt(ll x){if(x<0) putchar('-'),x=-x;if(x>9)
pt(x/10);putchar(x%10+'0');}

void print(ll x){pt(x), puts("");}

const ll mod = 1000000007;


ll fac[10000010];
ll ifac[10000010];
ll ksm(ll a,ll b){
    ll ans = 1;
    while(b){
        if(b&1)ans = ans * a % mod;
        a = a * a % mod;
        b >>= 1;
    }
    return ans;
}
ll C(ll a,ll b){
    return fac[a] * ifac[b] % mod * ifac[a-b] % mod;
}



void solve(){
    ll n=read(),k=read();
    if (k<2||n%2!=0){
        print(0);
    }
    else if(k>n/2){
        print(C(n, k));
    }
    else{
```

```
        print((C(n, k) - ksm(2,k)*C(n/2, k)%mod + mod)%mod);
    }
}

int main() {
    fac[0] = ifac[0] = 1;
    for(int i = 1; i <= 1e7; i++){
        fac[i] = fac[i-1] * i % mod;
    }
    ifac[10000000] = ksm(fac[10000000], mod-2);
    for(int i = 1e7-1; i >= 1; i--){
        ifac[i] = ifac[i+1] * (i+1) % mod;
    }
    ll t = read();
    while(t--){
        solve();
    }
    return 0;
}
```

D.暖鸭

[D-暖鸭_牛客练习赛132]

原题

题目

还记得什么是阶乘吗，现在不考这个...

问给定 T 组数据，每组数据给定 l,r

在所有的$l \le a \le b \le r$ 中，等可能地随机取一对(a, b)

求区间 [a, b] 至少包含一个平方数的概率

为了计算方便，最终答案乘上 (r-l+1)*(r-l+2) 后对 1000000007取模

输入

第一行输入一个 T，表示 T 组数据

接下来每行一组 l,r 如题所示$1 \le T \le 3 \times 10^5; 1 \le l \le r \le 1 \times 10^{16}$

输出

输出一个非负整数表示最终答案

示例1

输入

3
 1 1
 1 2
 1 3

输出

2
 4
 6

解释

[1,2] 中 a,b 有 [1,1], [1, 2], [2, 2]三种等可能区间，仅仅 [2, 2]没有包含完全平方数1

故答案为 2/3

输出 (2/3 * 3 * 2)%(1e9+7) = 4

题解

题目所求值为[a,b]内包含平方数的区间的两倍

不妨先求[a, b]内不包含平方数的区间数量

观察[1, 10]这个区间，枚举可得出不包含平方数的区间为区间[2, 3], [5, 8], [10,10]的子区间

可以发现这些区间是以平方数分割的

令$f(l,r)$为[l, r]内不包含平方数区间的个数，$h(x)$为长度为x的区间的子区间数量(令h(0)=0)

有

$$f(x^2,(x1)^2)=\frac{((x1)^2-x^2-1)*((x1)^2-1x^21))}{2}=\frac{2x(2x1)}{2}=x(2x1)$$
$$f(1,x^2)=\sum_{t=1}^{x-1}t\cdot(2t1)=\frac{(t-1)\cdot t\cdot(2t-1)}{3}\frac{(t-1)\cdot t}{2}$$
$$h(x)=\frac{x\cdot(x1)}{2}$$

令整数a, b 满足 $a\leqslant l^2,r^2\leqslant b$(可用二分查找求)

则答案为$h(b-a1)-f(a,b)$

$f(a,b)=f(a,l^2-1)f(l^2,r^2)f(r^21,b)$，有以下结果

1. $[a,l^2-1]$以及$[r^21,b]$内必定没有平方数，所以
   $f(a,l^2-1)f(r^21,b)=h(l^2-a)h(b-r^2)$

(当a=1, b=1时， l=1, r=1, 此时 $l^2-1=0<l,r^21=2>r$, 但右式答案仍正确,实际使用时直接计算即可)

2 .$f(l^2,r^2)=f(1,r^2)-f(1,l^2)$

最终式子为

$$h(b-a1)-(h(l^2-a)h(b-r^2)f(1,r^2)-f(1,l^2))$$
$$=\ (\frac{(b-a1)(b-a2)}{2})\ -\ ((\frac{(r-1)\cdot r\cdot(2r-1)}{6}\frac{(r-1)\cdot r}{4})\ -$$
$$(\frac{(l-1)\cdot l\cdot(2l-1)}{6}\frac{(l-1)\cdot l}{4}))\frac{(l^2-a1)(l^2-a2)}{2}\frac{(b-r^21)(b-r^22)}{2})$$

如果发现自己样例测试全部通过提交后仍为wa，建议带入10^{16}测试一下每个部分会不会溢出（最保险的就是求一步取一次模）

代码如下

```
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;
const ll MOD = 1000000007;

inline ll read(){
    ll s = 0, w = 1; char ch = getchar();
    while (ch < 48 || ch > 57) { if (ch == '-') w = -1; ch = getchar(); }
    while (ch >= 48 && ch <= 57) s = (s << 1) + (s << 3) + (ch ^ 48), ch =
getchar();
    return s * w;
}

inline void pt(ll x){if(x<0) putchar('-'),x=-x;if(x>9)
pt(x/10);putchar(x%10+'0');}

void print(ll x){pt(x), puts("");}

ll ksm(ll a,ll b){
    ll ans = 1;
    while(b){
        if(b&1)ans = ans * a % MOD;
        a = a * a % MOD;
        b >>= 1;
    }
    return ans%MOD;
}

ll sqrt(ll a)
{
    ll l=0,r=1e9;
    while(l<r){
        ll mid=(l+r)>>1;
        if(mid*mid<=a){
            l=mid;
        }
        else{
            r=mid-1;
        }
```

```
        if(l==r-1){
            if(r*r<=a){
                l=r;
            }
            break;
        }
    }
    return l;
}

ll g(ll x) {
    //f(1, x*x)
    x %= MOD;
    ll l = (x - 1 + MOD) % MOD;
    l = (l * x) % MOD;
    l = (l * (2 * x + MOD - 1) % MOD) % MOD;
    l = l * ksm(3,MOD-2) %MOD;
    ll r = (x - 1 + MOD) % MOD;
    r = (r * x) % MOD;
    r = r * ksm(2,MOD-2) % MOD;

    return (l + r) % MOD;
}

ll h(ll x){
    //长度x的区间的子区间数量
    ll l = x%MOD;
    l = l * ((x+1)%MOD) %MOD;
    l = l * ksm(2, MOD-2)%MOD;
    return l;
}

void solve(){
    ll a = read(), b = read();
    ll l = sqrt(a-1)+1, r = sqrt(b);
    if (r<l){
        print(0);
    }
    else{
        print((((h(b-a+1)- h(l*l-a) - h(b-r*r) - (g(r)-
g(l)))%MOD+MOD)*2+MOD)%MOD);
    }
}

int main(){
    ll T = read();
    while(T--){
        solve();
    }
}
```

E.先知

原题

题目

在n*m大小的二维矩阵中每个格子的初始高度为 a_{ij}

俗话说万丈高楼平地起...

你决定使用魔法：对于每个格子 a_{ij} 如果其四周(上下左右)均有格子，且四周格子高度均大于等于 a_{ij} 时实际高度会 +1，即 $a_{ij} = a_{ij}1$

你将会夜以继日充不停息，从上到下，从左到右遍历整个矩阵对于每个格子均使用一次魔法

但是，慢慢的，你好像发现时间足够长久后，貌似整个矩阵高度均不会发生变化

求最终矩阵的形态；为了输出方便，**请输出全部 a_{ij} 的和以及 a_{ij} 的异或和**

输入

第一行输入 n，m 两个整数表示 n行m列的矩阵

接下来 n 行，每行 m 个 a_{ij} 表示初始平地的高度

$$1 \le n, m \le 2 \times 10^3; 1 \le a_{ij} \le 1 \times 10^9$$

输出

输出一行两个整数

分别表示最终状态下 a_{ij} 的和 以及 a_{ij} 的异或和

示例1

输入

4 4

2 4 2 2

3 2 2 2

2 2 2 2

2 2 2 2

输出

40 0

说明

第一次遍历：

遍历到第 2 行第 2 列的数字2

有2<=min(4,3,2,2),

故变为

2 4 2 2

3 **3** 2 2

2 2 2 2

2 2 2 2

遍历到第 2 行第 3 列的数字2，

变为

2 4 2 2

3 3 **3** 2

2 2 2 2

2 2 2 2

遍历到第 3 行第 2 列的数字2，变为

2 4 2 2

3 3 3 2

2 **3** 2 2

2 2 2 2

遍历到第 3 行第 3 列的数字2，变为

2 4 2 2

3 3 3 2

2 3 **3** 2

2 2 2 2

第二次遍历：

遍历到第 2 行第 2 列的数字3 有3<=min(4,3,3,3),

故变为

2 4 2 2

3 **4** 3 2

2 3 3 2

2 2 2 2

此后不会有任何改变

和为 2+4+2+2+3+4+3+2+2+3+3+2+2+2+2+2=40

异或和为2^4^2^2^3^4^3^2^2^3^3^2^2^2^2^2=0

题解

不难发现， 一个点的值最大能到的值取决于周围最小值

只需维护所有点中的最小值及其位置（优先队列）， 每次取出最小值更新周围即可

全部更新完后遍历求答案

```
#include <bits/stdc++.h>
```

```
using namespace std;

typedef long long ll;
typedef pair<long long,long long> PLL;
typedef tuple<ll,ll,ll> TLLL;

inline ll read(){
    ll s = 0, w = 1; char ch = getchar();
    while (ch < 48 || ch > 57) { if (ch == '-') w = -1; ch = getchar(); }
    while (ch >= 48 && ch <= 57) s = (s << 1) + (s << 3) + (ch ^ 48), ch =
getchar();
    return s * w;
}

ll maze[2009][2009];
ll vis[2009][2009];


struct cmp {
    bool operator()(TLLL x, TLLL y) {
        return get<2>(x) > get<2>(y);
    }
};

int main(){
    ll n = read(), m = read();
    for(ll i=0;i<n;i++){
        for(ll j=0;j<m;j++){
            maze[i][j] = read();
        }
    }
    priority_queue<TLLL, vector<TLLL>, cmp> q;
    for(ll x=0;x<n;x++){
        for(ll y=0;y<m;y++){
            if ((x==0||x==n-1)||(y==0||y==m-1)){
                q.push(TLLL{x, y, maze[x][y]});
                vis[x][y] = 1;
            }
        }
    }

    vector<PLL> move = {{0, 1}, {0, -1}, {1, 0}, {-1 ,0}};
    while(!q.empty()){
        TLLL t = q.top();
        q.pop();
        for(PLL mo:move){
            ll x = get<0>(t) + mo.first;
            ll y = get<1>(t) + mo.second;
            if(1<=x&&x<=n-2&&1<=y&&y<=m-2&&!vis[x][y]){
                maze[x][y] = max(maze[x][y], get<2>(t) + 1);
                q.push(TLLL{x, y, maze[x][y]});
                vis[x][y] = 1;
            }
        }

    }
```

```
    ll ans1=0;
    ll ans2=0;
    for(ll i=0;i<n;i++){
        for(ll j=0;j<m;j++){
            ans1 += maze[i][j];
            ans2 ^= maze[i][j];
        }
    }
    cout<< ans1<<' '<<ans2<<'\n';
}
```

F.。

[F-。 牛客练习赛132]

原题

题目

多组样例 T

每输入三个数 a,b,c 表示木棒长度

你可以对任意边多次花费 1 的代价将某条边长度 +1 或 −1

使得最终的 a,b,c 可以组成直角三角形

求最小代价

输入

第一行输入 T 表示 T 组样例

每组样例输入三个正整数 a,b,c

$$1 \le T \le 20; 1 \le a, b, c \le 2 \times 10^5$$

输出

每组样例一个整数表示答案

如果无法构成直角三角形输出 −1

题解

a,b,c的范围很小， 可以预处理所有的组成直角三角形的三元组（即毕达哥拉斯三元组）

代码如下

```
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;
typedef tuple<ll,ll,ll> TLLL;
const ll inf =  0x3f3f3f3f;
```

```
inline ll read(){
    ll s = 0, w = 1; char ch = getchar();
    while (ch < 48 || ch > 57) { if (ch == '-') w = -1; ch = getchar(); }
    while (ch >= 48 && ch <= 57) s = (s << 1) + (s << 3) + (ch ^ 48), ch =
getchar();
    return s * w;
}

inline void pt(ll x){if(x<0) putchar('-'),x=-x;if(x>9)
pt(x/10);putchar(x%10+'0');}

void print(ll x){pt(x), puts("");}


vector<TLLL> findAll() {
    vector<TLLL> all;
    ll limit = 500000;
    for (ll m = 2; m * m <= limit; ++m) {
        for (ll n = 1; n < m; ++n) {
            if ((m - n) % 2 == 1 && gcd(m, n) == 1) {//生成本原毕达哥拉斯三元组
                ll a = m * m - n * n;
                ll b = 2 * m * n;
                ll c = m * m + n * n;

                ll mn = min({a, b, c}), mx = max({a, b, c});
                b = a + b + c - mn - mx;
                a = mn;
                c = mx;
                if (c > limit) break;
                for (ll k = 1; k * c <= limit; ++k) {
                    all.emplace_back(k * a, k * b, k * c);
                }
            }
        }
    }

    return all;
}


vector<TLLL> all;

void solve(){
    ll a = read(),b=read(),c=read();
    ll ans = inf;
    ll mn = min({a, b, c}), mx = max({a, b, c});
    b = a + b + c - mn - mx;
    a = mn;
    c = mx;
    for (const auto &triple : all) {
        ll na = get<0>(triple);
        ll nb = get<1>(triple);
        ll nc = get<2>(triple);
        ans = min(ans, abs(na - a) + abs(nb - b) + abs(nc - c));
     }
```

```
    print(ans);
}


int main() {
    all = findAll();
    ll T=read();
    while(T--){
        solve();
    }
    return 0;
}
```